# Array in Java

# Introduzione

Fino a questo punto abbiamo visto come dichiarare variabili di tipo primitivo e oggetti, gestire lo **scope**, usare cicli e strutture condizionali.
Tuttavia, molto spesso capita di dover memorizzare **più valori dello stesso tipo** e di volerli gestire in modo ordinato e indicizzabile.

## Gli array in Java

In Java, questo compito è svolto dagli **array**, un tipo speciale di struttura dati che permette di raggruppare più variabili dello stesso tipo sotto un unico nome.
Gli array in Java hanno alcune caratteristiche fondamentali:

- **Sono oggetti, non tipi primitivi, anche quando contengono valori primitivi.**
- **La dimensione dell'array è fissa e viene definita al momento della creazione.**
  - Non è possibile modificarla successivamente.
- **Ogni array contiene valori omogenei, cioè dello stesso tipo, che può essere sia un tipo primitivo (`int`, `double`, `boolean`, ecc.) sia un oggetto.**

## Dichiarazione degli array

Per dichiarare un array, bisogna indicare:

1. Il **tipo di elementi** che conterrà
2. Il nome della variabile array

Esempi di dichiarazione:

Java

```java
int[] arrayOfInt;     // stile più moderno e consigliato
int arrayOfInt[];     // stile "vecchio", simile al C
```

- La prima forma (`int[] arrayOfInt`) è:

    - **quella più usata in Java moderno ed è più chiara, perché indica subito che si tratta di un array di `int`.**
- La seconda forma ( `int arrayOfInt[]` ):
    - **viene ereditata dal linguaggio C e funziona ancora, ma oggi è considerata meno leggibile.**

Gli array classici hanno alcune limitazioni importanti:

- **Dimensione fissa**:
    - **una volta creato, un array può contenere solo un numero prestabilito di elementi.**
    - Ad esempio, se immaginiamo il carrello di un e-commerce, usare un array classico significherebbe avere un “tetto massimo” di prodotti.
- **Tipi omogenei**:
    - **tutti gli elementi devono essere dello stesso tipo**.
    - **Non è possibile inserire interi, stringhe e oggetti diversi nello stesso array.**

Per definire un array, si scrive prima il tipo di dato seguito dalle parentesi quadre `[]` :

Java

```java
int[] arrayOfInt;
String[] arrayOfString;
float[] arrayOfFloat;
```

Gli array possono anche essere **multidimensionali:**

- **cioè contenere a loro volta altri array**, come nel caso delle matrici:

Java

```java
int[][] matrix;   // matrice di interi
```

In sintesi, un array in Java è :

- **un contenitore ordinato di elementi omogenei, indicizzabile tramite numeri interi**
  e rappresenta un passo importante verso strutture dati più complesse come **liste, matrici e collezioni** che vedremo in seguito.

# Creare un array

Per creare un array in Java si utilizza l'operatore `new`, **specificando il tipo degli elementi e la dimensione dell'array**.

```java
int[] array = new int[100]; // crea un array che contiene 100 numeri interi
```

Con questa istruzione l'array viene **istanziato**:

- viene allocata memoria sufficiente per contenere **100 elementi di tipo** `int`;
- tutti gli elementi dell'array vengono **inizializzati automaticamente** con i **valori di default** del tipo.

**Valori di default negli array**

Quando un array viene creato con `new`, i suoi elementi assumono automaticamente un valore iniziale:

- `0` **per i tipi numerici interi (** `int`, `short`, `byte`, `long` **)**
- `0.0` **per i tipi in virgola mobile (** `float`, `double` **)**
- `false` **per il tipo** `boolean`
- `'\u0000'` **per il tipo** `char`
- `null` **per i tipi riferimento (oggetti)**

Di conseguenza, **prima di qualsiasi assegnazione esplicita**, tutti gli elementi dell'array contengono già un valore valido.

## Inizializzazione abbreviata: array anonimo

Java consente una forma abbreviata di creazione e inizializzazione di un array, detta **sintassi dell'array anonimo**:

```java
int[] smallPrime = {2,3,5} //ometto la new
```

In questo caso:

- l'uso dell'operatore `new` è implicito;

- la dimensione dell'array viene dedotta automaticamente dal numero di elementi forniti;
- l'array viene creato e inizializzato in un'unica istruzione.

### ⓘ Quando usare un array anonimo

L'array anonimo è particolarmente utile quando:

- si conoscono **a priori** il numero degli elementi e i loro valori;
- l'array deve contenere **valori costanti**;
- non è necessario modificare frequentemente il contenuto.

Un esempio tipico è un array che rappresenta i **giorni della settimana** o altri insiemi di valori fissi.

## Accesso agli elementi

Gli elementi di un array sono accessibili tramite un **indice**.

- Per convenzione (e per definizione del linguaggio), gli indici vanno da: **0** a `lunghezza - 1`
- Tramite l'indice è possibile:
  - leggere un valore
  - stamparlo
  - modificarlo

Esempio di assegnazione dei valori a un array:

Java

```java
for (int i = 0; i < 100; i++) {
    array[i] = i;
}

```

In questo esempio:

- `i` rappresenta l'indice
- `array[i]` rappresenta l'elemento in posizione `i`
- a ogni cella dell'array viene assegnato il valore dell'indice stesso

```java
int[] array;
array = 3
```

## Proprietà `length`

Ogni array possiede la proprietà `length`:

- **indica il numero di elementi dell'array.**
- **È una proprietà read-only**
- La sua sintassi è:

```java
nomeArray.length
```

- Il valore di `length` è **fisso** e viene stabilito al momento della creazione dell'array
- Non può essere modificato successivamente

### Stampare un array

Sebbene un array sia un oggetto, la stampa diretta **non mostra il contenuto**:

```java
System.out.println(array);
```

Questa istruzione stampa solo una **rappresentazione interna dell'oggetto** (tipo + riferimento in memoria), non gli elementi.

## Stampa corretta degli elementi

Per visualizzare i valori contenuti nell'array è necessario iterare sugli elementi:

```java
for (int i = 0; i < array.length; i++) {
    System.out.println(array[i]);
}
```

### Il costrutto `for-each`

Il costrutto `for-each` (detto anche *enhanced for*) rappresenta un'evoluzione del ciclo `for` tradizionale ed è utilizzato per **scorrere sequenzialmente gli elementi di un array**.

- Non è necessario:
  - **dichiarare un indice**
  - **incrementare manualmente il contatore**
- L'iterazione avviene automaticamente su tutti gli elementi dell'array

Java

```java
int[] giorniMese = { 31, 28, 30 };

for (int giorni : giorniMese) {
    System.out.println(giorni);
}
```

In questo esempio:

- `giorni` **è una variabile che assume, a ogni iterazione, il valore dell'elemento corrente**
- `giorniMese` **è l'array da scorrere**

> ⓘ **Limiti del** `for-each`
>
> Il `for-each` presenta alcune limitazioni:
>
> - **scorre sempre dall'indice** `0` **all'indice** `n - 1`
> - non consente:
>  - **accesso diretto all'indice**
>  - **modifiche strutturali dell'array**
>  - **salti condizionati sugli indici**
>
> Per questi motivi, quando è necessario lavorare sugli indici, il ciclo `for` classico resta più adatto.

## Sfondamento dell'array

Se si tenta di accedere a un elemento **fuori dall'intervallo valido** degli indici, il programma genera un errore **a runtime**.

Esempio di errore:

```shell
ArrayIndexOutOfBoundsException
```

Questo avviene quando:

- **l'indice è negativo**
- **l'indice è maggiore o uguale alla lunghezza dell'array**

## Array multidimensionali

n Java è possibile creare **array multidimensionali**, che sono modellati come **array di array**.

- È possibile creare array a:
  - **2 dimensioni (matrici)**
  - **3, 4 o più dimensioni**
- **Ogni dimensione rappresenta un livello di annidamento**

Esempio di array bidimensionale (matrice):

```java
int[][] matrix = new int[3][4]; // 3 righe, 4 colonne
```

**Proprietà `length` negli array multidimensionali**

Per una matrice:

- `matrix.length` **indica il numero di righe**
- `matrix[i].length` **indica il numero di colonne della riga** `i`

### Accesso agli elementi di una matrice

Per accedere correttamente agli elementi di un array bidimensionale si utilizzano due cicli annidati:

```java
for (int i = 0; i < matrix.length; i++) {
```

```
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.println(matrix[i][j]);
    }
}

```

In questo esempio:

- `i` **rappresenta l'indice della riga**
- `j` **rappresenta l'indice della colonna**
- `matrix[i][j]` **identifica il singolo elemento della matrice**

## Matrici frastagliate (Jagged Array)

In Java è possibile creare **matrici frastagliate:**

- **ovvero array bidimensionali in cui le righe possono avere lunghezze diverse**.
  Questo è possibile perché, internamente, una matrice è modellata come un **array di array**.

### Dichiarazione e creazione

La matrice viene prima dichiarata specificando **solo il numero di righe**:

Java

```
int[][] matrix = new int[4][];
```

Successivamente, ogni riga viene inizializzata separatamente, assegnando un array di lunghezza diversa:

Java

```
matrix[0] = new int[3];
matrix[1] = new int[1];
matrix[2] = new int[4];
matrix[3] = new int[2];
```

A questo punto la matrice è completamente utilizzabile.

### Osservazioni sulla proprietà `length`

Nelle matrici frastagliate:

- `matrix.length` **indica il numero di righe**
- `matrix[i].length` **indica **la lunghezza della riga `i`**

Nel caso specifico:

- `matrix[0].length` → 3
- `matrix[1].length` → 1
- `matrix[2].length` → 4
- `matrix[3].length` → 2

mentre:

- `matrix.length` → 4

## Considerazioni

Le matrici frastagliate:

- **consentono un uso più flessibile della memoria**
- **sono utili quando le righe non hanno tutte lo stesso numero di elementi**
- **richiedono attenzione nell'accesso agli elementi, poiché ogni riga può avere una lunghezza diversa**

Per questo motivo, quando si scorre una matrice frastagliata, è fondamentale usare sempre:

Java

```java
matrix[i].length
```

e **non assumere una dimensione fissa delle colonne**.